

Advanced NLP

Session 03 · The Transformer Block

Prof. Dr. Richard Sieg · TH Köln IWS · WS 26 / 27
Tuesday, October 13, 2026

From one attention head to a block you can stack

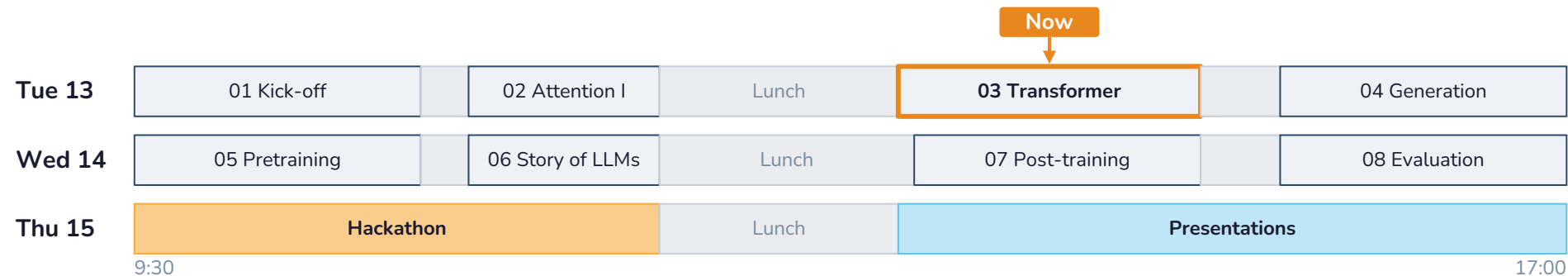
Time

13:30 Fixing self-attention: positions, the feed-forward layer, residuals, normalization

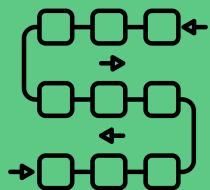
13:50 The Transformer: multi-head attention, the decoder, the mask

14:20 Lab: notebooks 02 and 03

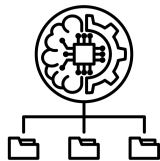
15:05 Break



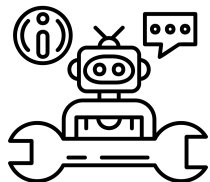
Three ingredients make an LLM



Process long sequences and context



Efficient training on huge datasets

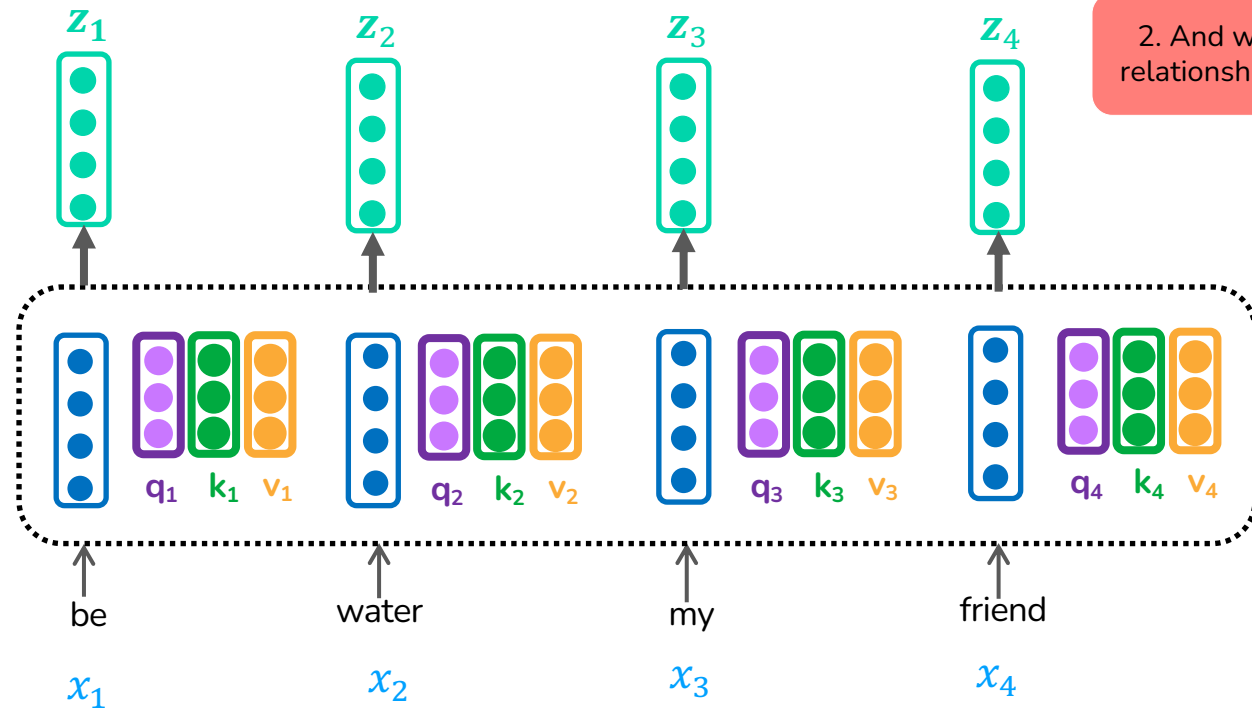


Follow (human) instructions

Where we left off: one head, two problems

1. But now we have no sense for the order of the tokens!

2. And we can only learn linear relationships between the tokens!



Self-Attention
Head

A Transformer block changes what a vector knows, never its shape.

Agenda

01. Fixing self-attention

02. Transformers

03. Tutorial

01.

Fixing self-attention

Fix 1: add a position vector to every token

- We do not have a concept for the position of a word (same as bag of words)
- **Solution:** add position vectors to the embedding inputs, $\tilde{x}_i = x_i + p_i$ where $p_i \in \mathbb{R}^d$ (**positional embedding**)
- These position vectors can be fixed or learned. GPT-2 learns them, and so does our model: a second table, one row per position, 512×256 .
- Examples of fixed position vectors
 - One-hot encodings $p_i = (0, \dots, 0, 1, 0, \dots, 0)$ (not useful in practice)
 - **Sinusoidal** (original paper uses this):

$$p_i = (\sin(i/10000^{2 \cdot 1/d}), \cos(i/10000^{2 \cdot 1/d}), \dots, \sin(i/10000^{2 \cdot d/2/d}), \cos(i/10000^{2 \cdot d/2/d}))$$

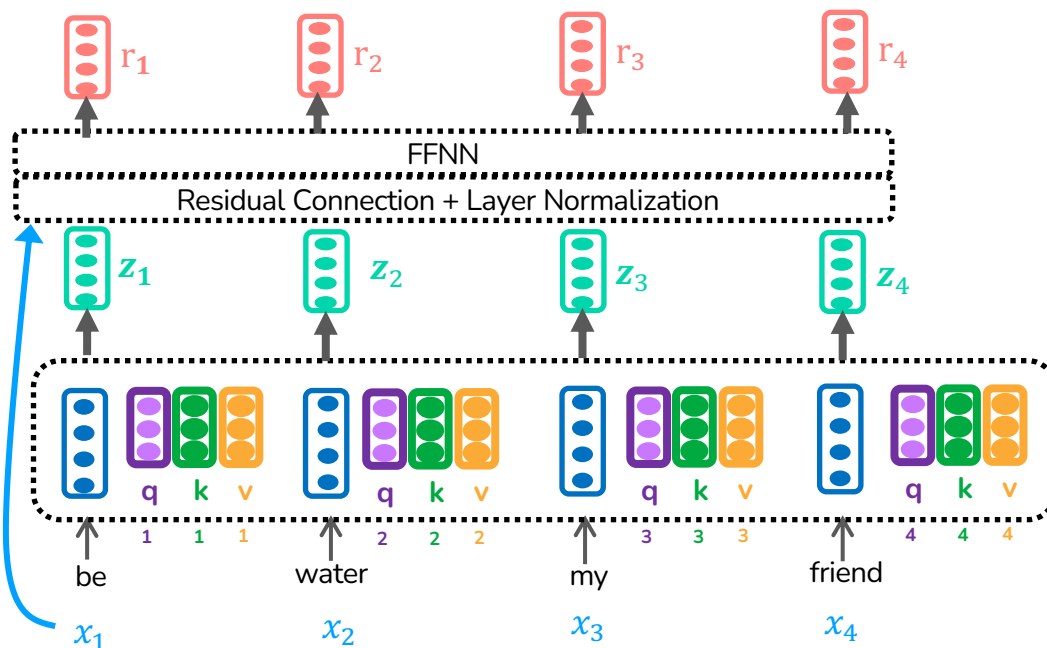
Fix 2: a feed-forward network on every token

GPT-2: GELU instead of ReLU

- Solution for non-linearity:**

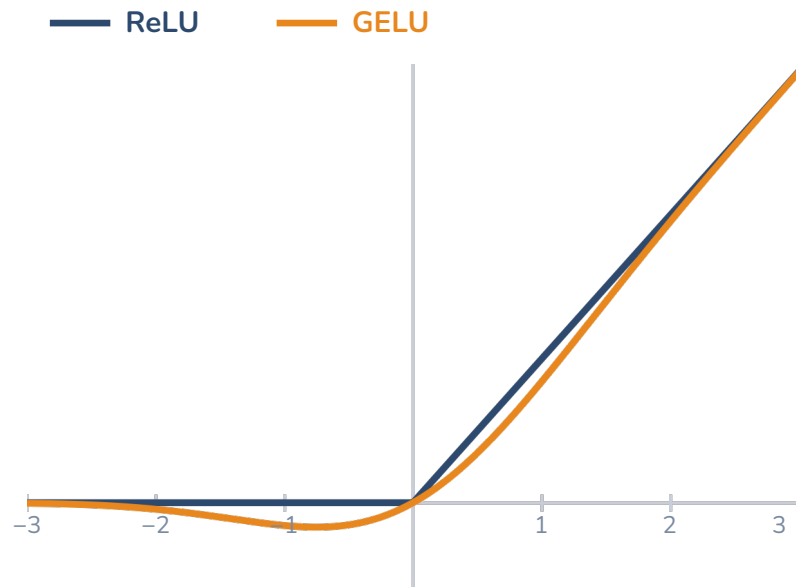
add a feed-forward neural network on each z_i : $W_2 \cdot \text{ReLU}(W_1 \cdot z_i + b_1) + b_2$

- Also, we add residual connections and layer normalization



GPT-2 uses GELU, a smooth version of ReLU

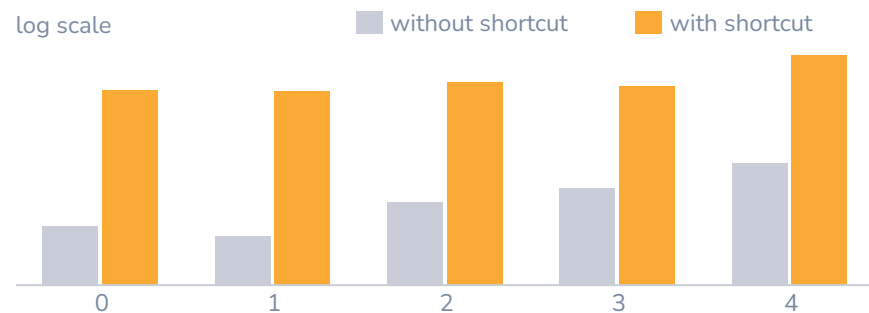
- $ReLU(x) = \max(0, x)$: zero for every negative input, and a zero gradient there too.
- $GELU(x) = x \cdot \Phi(x)$, with Φ the standard normal distribution function: close to ReLU for large inputs, smooth near zero, a small non-zero gradient for negative ones.
- GPT-2 was trained with a *tanh* approximation of GELU. Our notebook uses the same.
- The feed-forward layer expands by four and contracts back: 256 to 1,024 to 256 in our model.



Residual connections give the gradient a shortcut

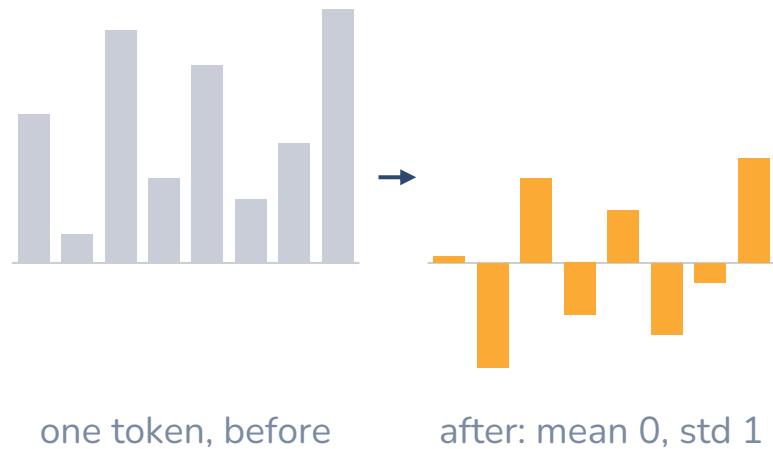
- Each sublayer's output is added to its own input: $x + f(x)$.
- The idea comes from residual networks in computer vision.
- Without it, gradients shrink layer by layer on their way back, and the first layers barely learn.
- Five layers, same seed, mean absolute gradient per layer:

Layer	Without shortcut	With shortcut
0, first	0.000202	0.222
1	0.000120	0.207
2	0.000715	0.329
3	0.001399	0.267
4, last	0.005050	1.326



LayerNorm rescales every token's vector on its own

- For each token: subtract the mean of its 256 numbers, divide by their standard deviation.
- Then a learned scale and shift, so the network can undo the normalization where it helps.
- Per token, not per batch: it works the same for a batch of one.
- It keeps activations in a stable range, however many blocks are stacked.



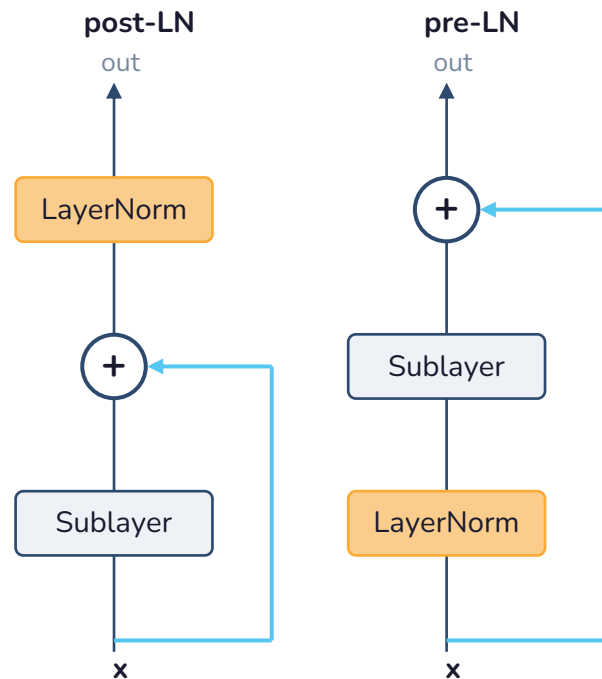
GPT-2 normalizes before each sublayer, not after

2017 Transformer,
post-LN

GPT-2, pre-LN

Order	sublayer, add x, then normalize	normalize, sublayer, then add x
Residual path	normalized after every block	runs straight through, untouched

- Pre-LN trains more stably and needs less learning-rate warmup.
- The 2017 diagrams on the next slides say “Add & Norm” after each sublayer. That is post-LN. Our code is pre-LN.



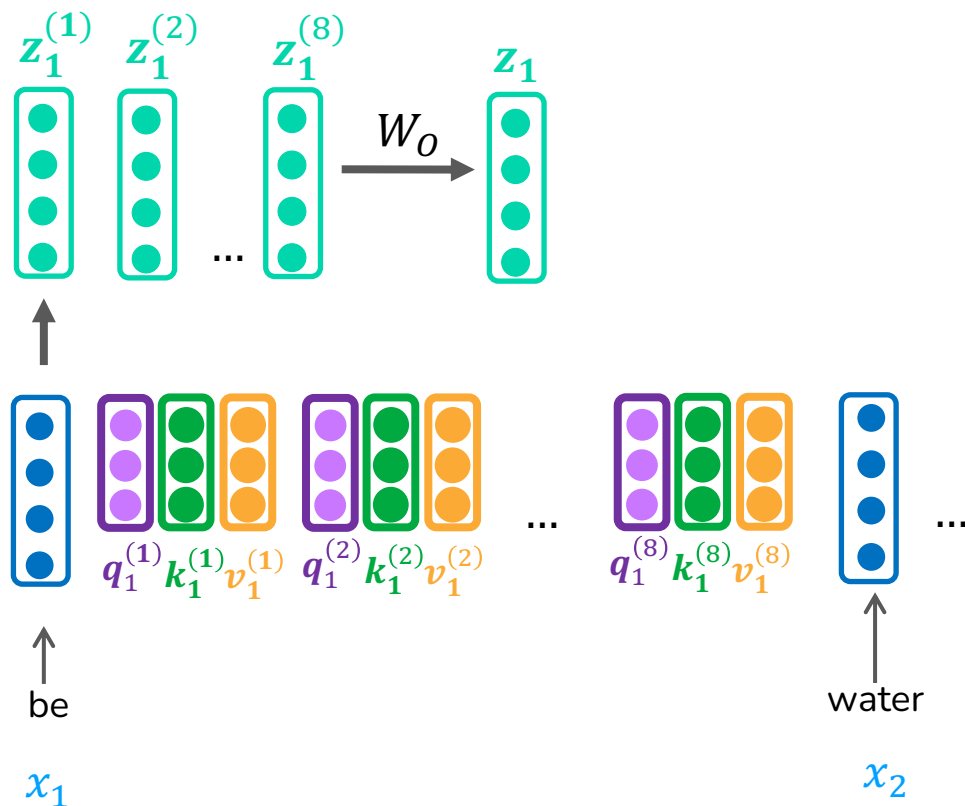
Let's stack things up!



02.

Transformers

Several heads look for different things in parallel



- We can use multiple weight matrices W_q, W_k, W_v in parallel
- This allows tokens to focus on different positions
- We call each set of these matrices an **attention head**
- The output z_i of each attention head is then concatenated $[z_1, \dots, z_h]$ and then projected back to the original dimension with a weight matrix W_o

Eight heads of 32

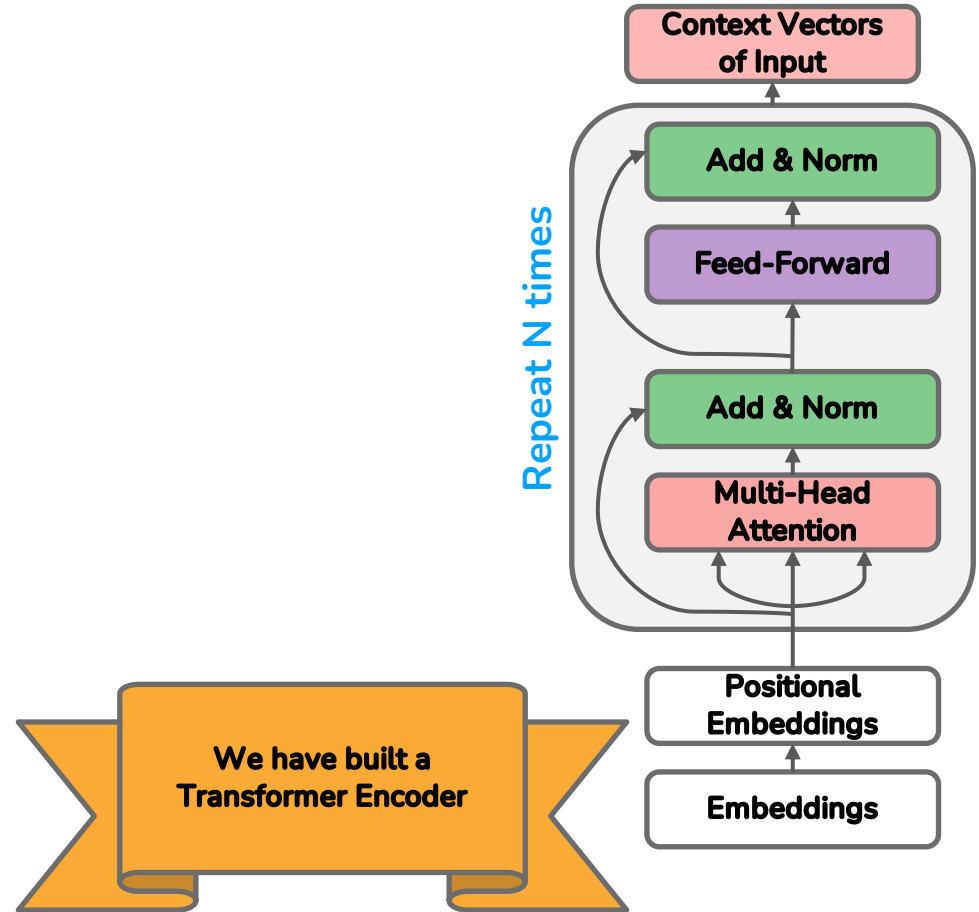
Step	Shape
Input x	(b, 512, 256)
One projection each for queries, keys, values	(b, 512, 256)
Split into 8 heads of 32	(b, 512, 8, 32)
Heads next to the batch	(b, 8, 512, 32)
Scores per head, divided by $\sqrt{32}$	(b, 8, 512, 512)
Weighted values, heads put back together	(b, 512, 256)
Output projection W_o	(b, 512, 256)

- The same computation as eight separate small W_q, W_k, W_v , done in one matrix product.

(b, 512, 256) in, (b, 512, 256) out. That is why blocks can be stacked.

Stacking blocks gives a Transformer encoder

- Now we can stack multiple Transformer Blocks on top of each other.
- Each block has:
 - Multi-head attention
 - Residual connections and layer normalization
 - Feed-Forward Layer
 - Another layer of residual connections and normalization



The decoder may not look into the future

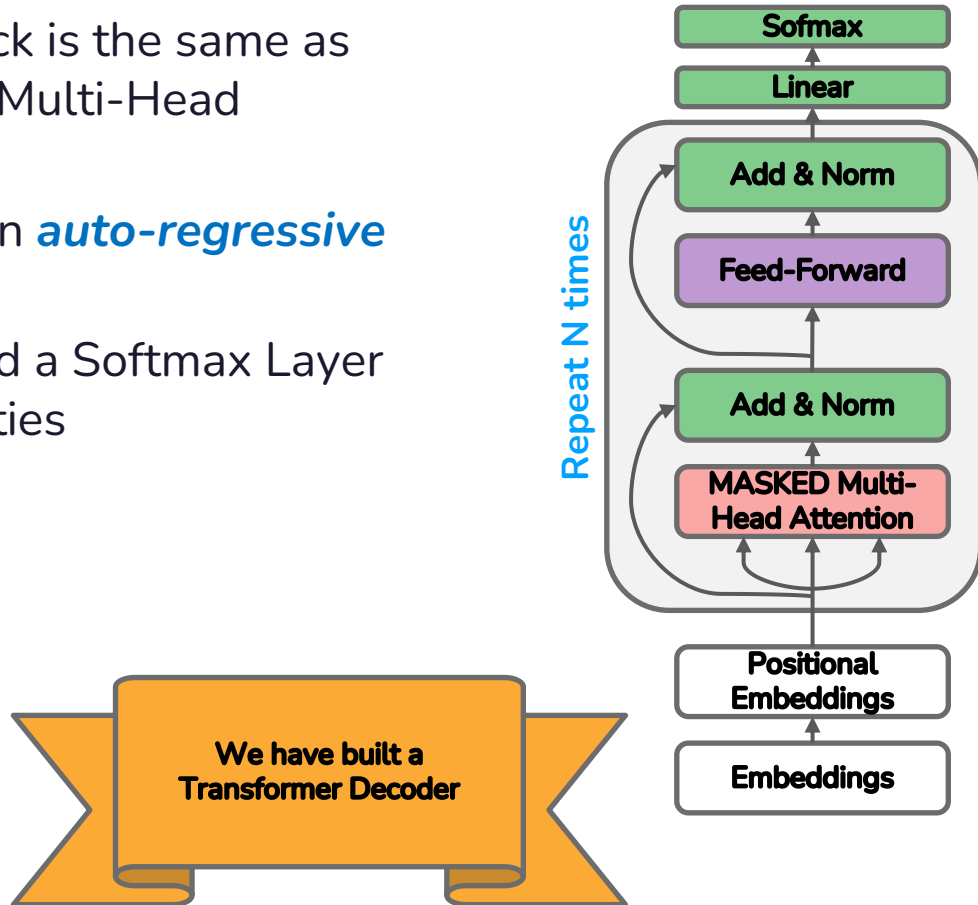
- The Decoder has to generate the output sequence
- So, during training we need to make sure that the Decoder can't look into the future
- Solution: mask out the attention by setting the attention of future words to $-\infty$
→ get 0 weight in softmax
- Our attention matrix becomes

$$e_{ij} = \begin{cases} q_i^T k_j, j \leq i \\ -\infty, j > i \end{cases}$$

	<Start>	be	water	my	friend
<START>		$-\infty$	$-\infty$	$-\infty$	$-\infty$
be			$-\infty$	$-\infty$	$-\infty$
water				$-\infty$	$-\infty$
my					$-\infty$
friend					

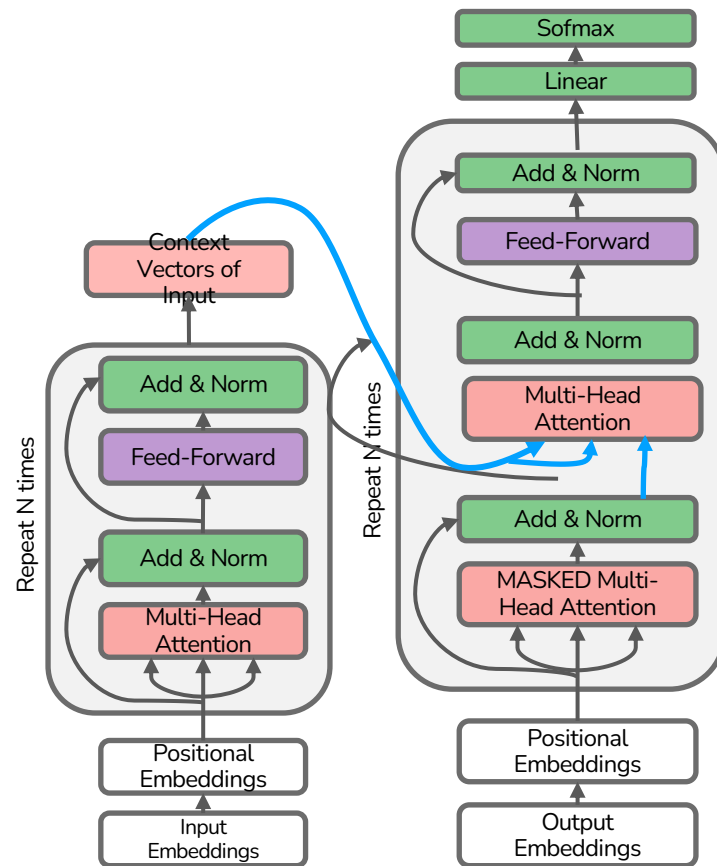
A decoder is the same block with a mask

- The Transformer Decoder Block is the same as the Encoder just with Masked Multi-Head Attention
- So, the Decoder performs as an *auto-regressive* Language Model
- In the end, we add a Linear and a Softmax Layer to get the next word probabilities



Cross-attention joins the encoder and the decoder

- For seq2seq tasks like Machine Translation, we can couple the Encoder and Decoder
- For this we add another Multi-Head Attention Block to the Decoder that receives the output vectors $z_1, \dots, z_n$ from the Encoder (**Cross-Attention**)
- In this block, the keys and values are computed using the Encoder output vectors and the queries are computed using the Decoder input vectors $h_1, \dots, h_n$
- $k_i = W_k z_i, v_i = W_v z_i, q_i = W_q h_i$



Which Model for Which Task?

Discriminative Task

Encoder Model

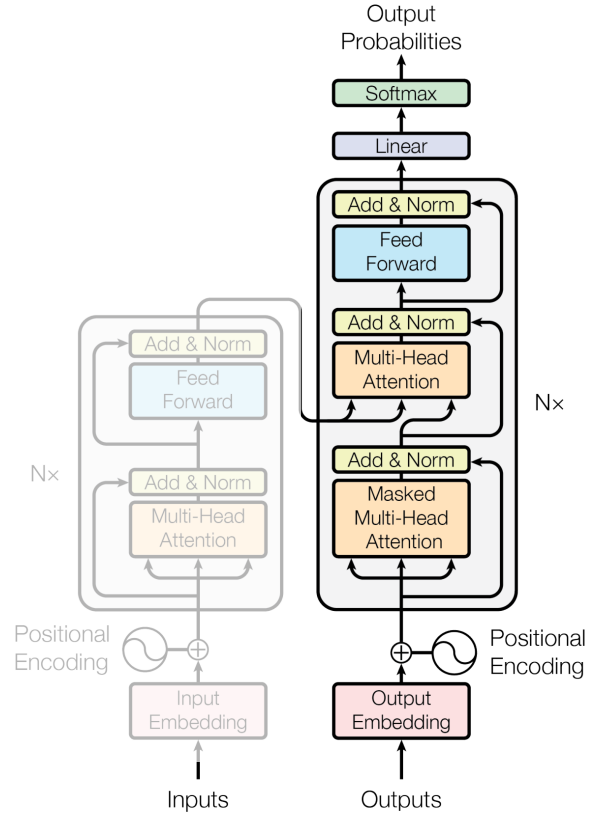
Language Modeling

Decoder Model

LLMs

Seq2Seq

Encoder - Decoder
Model



Most of our parameters sit in the token table

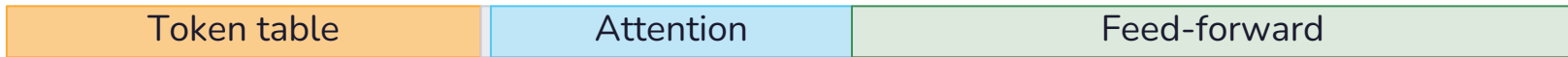
	Our model, 17.7M	GPT-2, 124M
Token embedding table	73%	31%
Positional table	1%	1%
Attention, all blocks	9%	23%
Feed-forward, all blocks	18%	46%

- 73 percent of our 17.7 million parameters are the token table. The six blocks together are about a quarter.
- In GPT-2 the blocks dominate, and the feed-forward layers most of all. Scale changes where the weight sits.

Our model



GPT-2



A Transformer block changes what a vector knows, never its shape.

Almost every piece has a modern replacement

Built today

LayerNorm, before each sublayer

Feed-forward layer with GELU

Learned positions

Full causal attention

Scores from queries and keys

Multi-head attention

Generation (covered in next block)

On Thursday

RMSNorm, and norm placement

SwiGLU

sinusoidal positions, RoPE

sliding-window attention

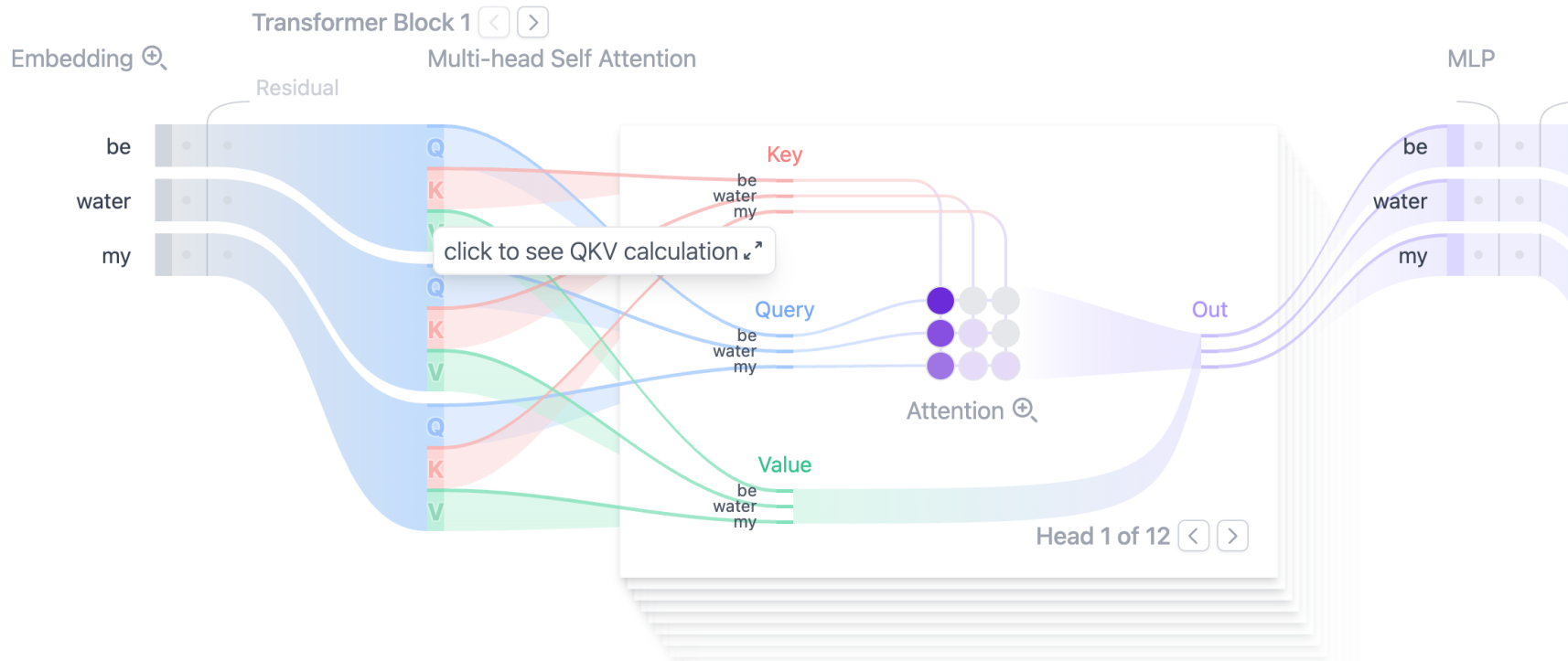
QK-Norm

grouped-query attention

KV cache

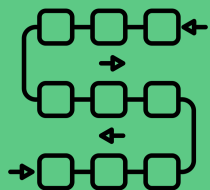
On Wednesday evening, one of these becomes yours.

Folks do great visualizations of the transformer

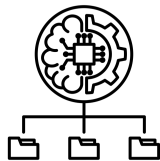


Transformer Explainer

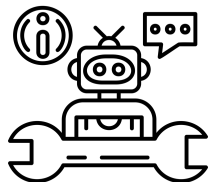
Three ingredients make an LLM



Process long sequences and context



Efficient training on huge datasets



Follow (human) instructions

03.

Tutorial

Notebooks 02 and 03: build attention, then the block

1. Notebook 02: implement `MultiHeadAttention`. The checks below the class turn green when it works.
2. Notebook 03: implement `LayerNorm`, `FeedForward` and `TransformerBlock`, in this order.
3. `TransformerBlock` imports your `MultiHeadAttention` from notebook 02, so it only runs once step 1 works.
4. Run notebook 03 top to bottom. Every block returns the shape it was given.
5. Commit and push before 15:00.

- `MultiHeadAttention` passes its checks
- `TransformerBlock` turns a (2, 5, 16) input into a (2, 5, 16) output
- Your work is pushed

About 45 minutes, in pairs

Sources

Literature

- Raschka, Build a Large Language Model (From Scratch), chapter 3, sections 3.5 and 3.6: causal attention, dropout, multi-head attention with one split projection. Chapter 4, sections 4.2 to 4.5: LayerNorm, GELU and the feed-forward layer, shortcut connections, the transformer block, pre-LN. Listing 4.5 for the gradient table.
- Vaswani et al. 2017, Attention Is All You Need, arxiv.org/abs/1706.03762, section 3.5 and figure 1.
- He et al. 2015, Deep Residual Learning for Image Recognition, arxiv.org/abs/1512.03385.

Literature, continued

- Ba, Kiros, Hinton 2016, Layer Normalization, arxiv.org/abs/1607.06450.
- Hendrycks, Gimpel 2016, Gaussian Error Linear Units, arxiv.org/abs/1606.08415.
- Xiong et al. 2020, On Layer Normalization in the Transformer Architecture, arxiv.org/abs/2002.04745.

Tools

- Transformer Explainer, poloclub.github.io/transformer-explainer

Sources, checked numbers

Checked numbers, as of October 2, 2026

- Parameter breakdown, computed from the configurations: our model 17,731,328 in total, token table 12,865,792, positional table 131,072, attention 1,574,400, feed-forward 3,153,408. GPT-2 124,439,808 in total, feed-forward 56,669,184. Both totals match the known sizes.
- Shapes: `emb_dim` 256, 8 heads of 32, context 512, from the course configuration.
- Gradients with and without shortcuts: Raschka's listing 4.5, run on October 2, 2026 with torch 2.14.0, seed 123.